

从根上理解Redis分布式锁演进架构

分布式锁相信大家一定不会陌生，想要用好或者自己写一个却没那么简单。

想要达到上述的条件，一定要 **掌握分布式锁的应用场景**，以及分布式锁的不同实现，不同实现之间有什么区别。

分布式锁场景

如果想真正了解分布式锁，需要结合一定场景；举个例子，某夕夕上抢购 AirPods Pro 的 100 元优惠券。

优惠详情



领取优惠券

商品券

¥100

仅限该商品使用
领券后1小时内有效

12:00开抢

如果使用下面这段代码当作抢购优惠券的后台程序，我们一起看一下，可能存在什么样的问题。

```
@Slf4j @RestController
public class DiscountController {
    @Autowired
    private StringRedisTemplate cache;

    @GetMapping("/acquire")
    public String acquireDiscount() {
        String lockKey = "surplus";
        Object surplusObj = cache.opsForValue().get(lockKey);
        Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

        if (surplusNum <= 0) return "fail";

        // ...执行逻辑操作
        // 对优惠券剩余数量 -1
        surplusNum -= 1;
        // 更新 cache
        cache.opsForValue().set(lockKey, surplusNum.toString());
        return "success";
    }
}
```

很明显的就是这段流程在并发场景下并不安全，会导致优惠券发放超过预期，类似电商抢购超卖问题。

想一哈有什么方式可以避免这种分布式下超量问题？

互斥加锁，Java 中互斥锁的语义就是 **同一时间，只允许一个客户端对资源进行操作**。

比如 Java 中的关键字 **Synchronized**，以及 JUC Lock 包下的 ReentrantLock 都可以实现互斥锁。

JVM 本地锁

如图所示，加入 JVM synchronized 锁确实可以解决单机下并发问题。

```
@GetMapping("/acquire")
public String acquireDiscount() {
    synchronized (this) {
        String lockKey = "surplus";
        Object surplusObj = cache.opsForValue().get(lockKey);
        Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

        if (surplusNum <= 0) return "fail";

        // ...执行逻辑操作
        // 对优惠券剩余数量 -1
        surplusNum -= 1;
        // 更新 cache
        cache.opsForValue().set(lockKey, surplusNum.toString());
    }
    return "success";
}
```

但是生产环境为了保证服务高可用，起码要 **部署两台服务**，这样的话 `synchronized` 就不起作用了，因为它的 **作用域只是单个 JVM**。

分布式情况下只能通过 **分布式锁** 来解决多个服务资源共享的问题了。

分布式锁

分布式锁的定义：**保证同一时间只能有一个客户端对共享资源进行操作。**

比对刚才举的例子，不论部署多少台优惠券服务，只会有 **一台服务能够对优惠券数量进行增删操作**。

另外有几点要求也是必须要满足的：

1. **不会发生死锁**。即使有一个客户端在持有锁的期间崩溃而没有主动解锁，也能保证后续其他客户端能加锁。
2. **具有容错性**。只要大部分的 Redis 节点正常运行，客户端就可以加锁和解锁。
3. **解铃还须系铃人**。加锁和解锁必须是同一个客户端，客户端自己不能把别人加的锁给解了。

分布式锁实现大致分为三种，Redis、Zookeeper、数据库，文章以 Redis 展开分布式锁的讨论。

分布式锁演进史

先来构思下分布式锁实现思路。

首先我们必须保证同一时间只有一个客户端(部署的优惠券服务)操作数量加减。其次本次 **客户端操作完成后**，需要让 **其它客户端继续执行**：

1. 客户端一存放一个标志位，如果添加成功，操作减优惠券数量操作。
2. 客户端二添加标志位失败，本次减库存操作失败（或继续尝试获取等）。
3. 客户端一优惠券操作完成后，需要将标志位释放，以便其余客户端对库存进行操作。

1. 第一版 setnx

向 Redis 中添加一个 lockKey 锁标志位，如果添加成功则能够继续向下执行扣减优惠券数量操作，最后再释放此标志位。

```
@GetMapping("/acquire")
public String acquireDiscount() {
    String lockKey = "lockKey"; // 设置锁标志位
    Boolean ifAbsent = cache.opsForValue().setIfAbsent(lockKey, "lock"); // 不存在即添加，存在返回 False
    if (!ifAbsent) return "fail"; // 未获取到分布式锁标志位，失败

    String surplus = "surplus";
    Object surplusObj = cache.opsForValue().get(surplus);
    Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

    if (surplusNum <= 0) return "fail"; // 库存数量不足，返回失败

    // ...执行逻辑操作
    surplusNum -= 1; // 对优惠券剩余数量 -1
    cache.opsForValue().set(surplus, surplusNum.toString()); // 更新库存 cache
    cache.delete(lockKey); // 删除分布式锁
    return "success";
}
```

由于使用的是 Spring 提供的 Redis 封装的 Start 包，所以有些命令与 Redis 原生命令不相符。

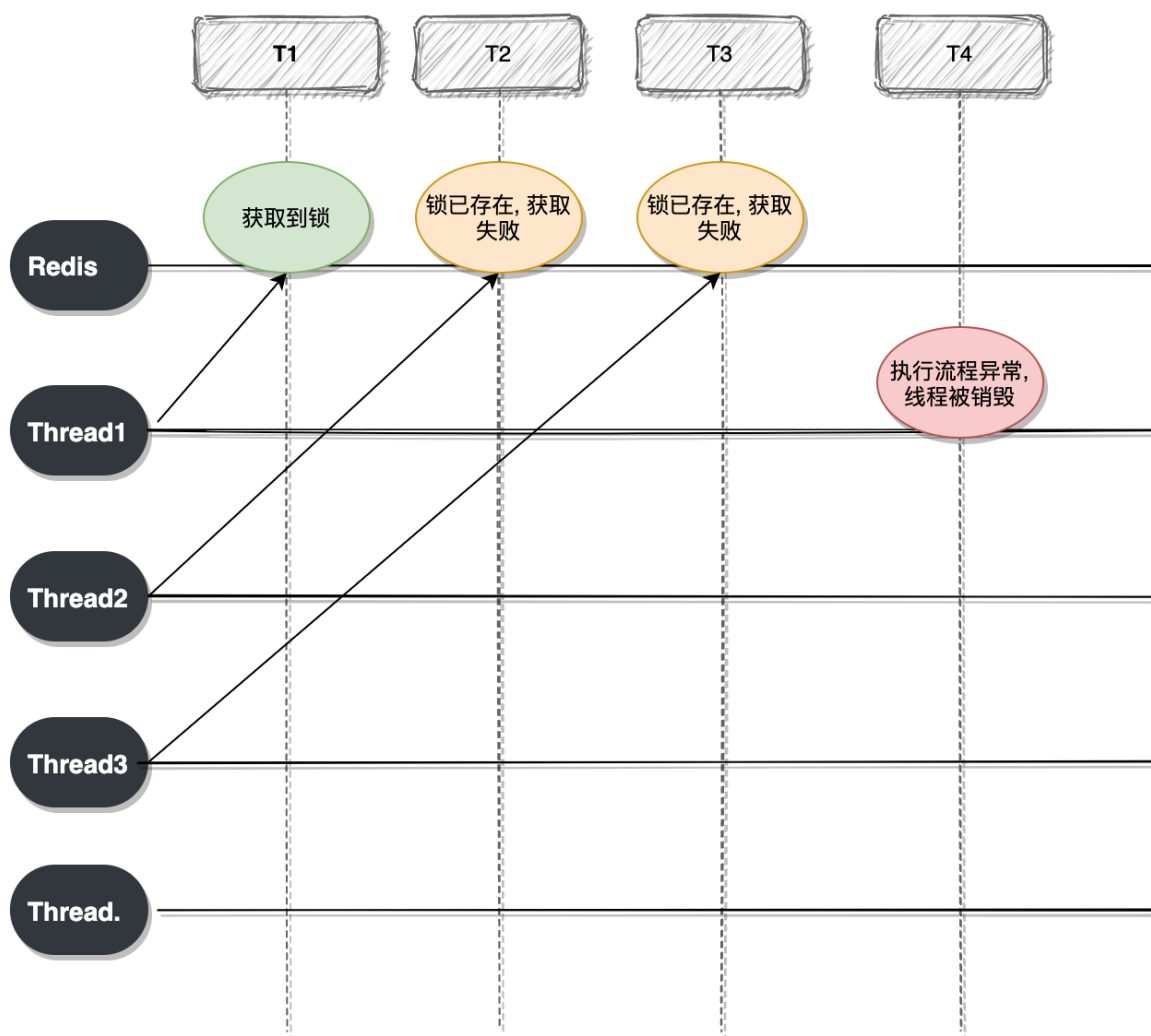
```
setIfAbsent(key, val) -> setnx(key, val)
```

加了简单的几行代码，一个简单的分布式锁的雏形就出来了。

2. 第二版 expire

上面第一版基于 setnx 命令实现分布式锁的缺陷也是很明显的，那就是 **一定情况下可能发生死锁**。

画个图，举个例子说明哈。



上图说明，线程 1 在成功获取锁后，执行流程时异常结束，**没有执行释放锁操作**，这样就会 **产生死锁**。

如果方法执行异常导致的线程被回收，那么可以将解锁操作放到 finally 块中。

但是还有存在死锁问题，**如果获得锁的线程在执行中，服务被强制停止或服务器宕机**，锁依然**不会得到释放**。

这种极端情况下我们还是要考虑的，毕竟不能只想着服务没问题对吧。

对 Redis 的 **锁标志位加上过期时间** 就能很好的防止死锁问题，继续更改下程序代码。

```
@GetMapping("/acquire")
public String acquireDiscount() {
    String lockKey = "lockKey"; // 设置锁标志位
    Boolean ifAbsent = cache.opsForValue().setIfAbsent(lockKey, "lock"); // 不存在即添加，存在返回 False

    if (!ifAbsent) return "fail"; // 未获取到分布式锁标志位，失败

    // 对分布式锁标志位添加过期时间
    cache.expire(lockKey, 5, TimeUnit.SECONDS);
    String surplus = "surplus";
    Object surplusObj = cache.opsForValue().get(surplus);
    Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

    if (surplusNum <= 0) return "fail"; // 库存数量不足，返回失败

    // ...执行逻辑操作
    surplusNum -= 1; // 对优惠券剩余数量 -1
    cache.opsForValue().set(surplus, surplusNum.toString()); // 更新库存 cache
    cache.delete(lockKey); // 删除分布式锁
    return "success";
}
```

虽然 **小红旗处** 对分布式锁添加了过期时间，但依然无法避免极端情况下的死锁问题。

那就是如果在客户端加锁成功后，还没有设置过期时间时宕机。

如果想要避免添加锁时死锁，那就对添加锁标志位 & 添加过期时间命令 **保证一个原子性**，要么一起成功，要么一起失败。

3. 第三版 set

我们的添加锁原子命令就要登场了，从 Redis 2.6.12 版本起，提供了可选的 **字符串 set 复合命令**。

```
SET key value [expiration EX seconds|PX milliseconds] [NX|XX]
```

可选参数如下：

- **EX**: 设置超时时间，单位是秒。
- **PX**: 设置超时时间，单位是毫秒。

- **NX**: IF NOT EXIST 的缩写，只有 **KEY 不存在的前提下** 才会设置值。
- **XX**: IF EXIST 的缩写，只有在 **KEY 存在的前提下** 才会设置值。

继续完善分布式锁的应用程序，代码如下：

```
@GetMapping("/acquire")
public String acquireDiscount() {
    Jedis cache = new Jedis("localhost", 6379);
    cache.auth("123456");
    String lockKey = "lockKey"; // 设置锁标志位
    // ▶ 重点哦！
    String setResult = cache.set(lockKey, "lock", "nx", "ex", 5); // 成功添加返回 OK

    if (!Objects.equals(setResult, "OK")) return "fail"; // 未获取到分布式锁标志位，失败

    String surplus = "surplus";
    Object surplusObj = cache.get(surplus);
    Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

    if (surplusNum <= 0) return "fail"; // 库存数量不足，返回失败

    // ...执行逻辑操作
    surplusNum -= 1; // 对优惠券剩余数量 -1
    cache.set(surplus, surplusNum.toString()); // 更新库存 cache
    cache.del(lockKey); // 删除分布式锁
    return "success";
}
```

我使用的 **2.0.9.RELEASE** 版本的 SpringBoot，RedisTemplate 中不支持 set 复合命令，所以临时换个 Jedis 来实现。

加锁以及设置过期时间确实保证了原子性，但是这样的分布式锁就没有问题了吗？

我们根据图片以及流程描述设想一下这个场景：

1. 线程一获取锁成功，设置过期时间五秒，接着执行业务逻辑；
2. 接着线程一获取锁后执行业务流程，执行的时间超过了过期时间，锁标志位过期进行释放，此时线程二获取锁成功；
3. 然而此时线程一执行完业务后，开始执行释放锁的流程，然后顺手就把线程二获取的锁释放了。

如果线上真的发生上述问题，就可能会造成线上数据和业务的运行异常，更甚者可能在线程一将线程二的锁释放掉之后，线程三获取到锁，然后线程二执行完将线程三的锁释放。

4. 第四版 verify value

事到如今，只能创建辨别客户端身份的唯一值了，将加锁及解锁归一化，上代码。

```
@GetMapping("/acquire")
public String acquireDiscount() {
    Jedis cache = new Jedis("localhost", 6379);
    cache.auth("123456");
    String lockKey = "lockKey"; // 设置锁标志位
    String lockValue = UUID.randomUUID().toString(); // 重点哦，设置客户端唯一标示
    String setResult = cache.set(lockKey, lockValue, "nx", "ex", 5); // 不存在即添加，存在返回 False

    if (!Objects.equals(setResult, "OK")) return "fail"; // 未获取到分布式锁标志位，失败

    String surplus = "surplus";
    Object surplusObj = cache.get(surplus);
    Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

    if (surplusNum <= 0) return "fail"; // 库存数量不足，返回失败

    // ...执行逻辑操作
    surplusNum -= 1; // 对优惠券剩余数量 -1
    cache.set(surplus, surplusNum.toString()); // 更新库存 cache

    // 重点哦，只有在确定身份后才会执行删除
    if (Objects.equals(cache.get(lockKey), lockValue)) cache.del(lockKey); // 删除分布式锁

    return "success";
}
```

这一版的代码相当于我们添加锁标志位时，同时为每个客户端设置了 uuid 作为锁标志位的 val，解锁时需要判断锁的 val 是否和自己客户端的相同，辨别成功才会释放锁。

但是上述代码执行业务逻辑如果抛出异常，锁只能等待过期时间，我们可以将解锁操作放到 finally 块。


```

@GetMapping("/acquire")
public String acquireDiscount() {
    Jedis cache = new Jedis("localhost", 6379);
    cache.auth("123456");
    String lockKey = "lockKey"; // 设置锁标志位
    String lockValue = UUID.randomUUID().toString(); // 重点哦，设置客户端唯一标示
    try {
        String setResult = cache.set(lockKey, lockValue, "nx", "ex", 5); // 不存在即添加，存在返回 False

        if (!Objects.equals(setResult, "OK")) return "fail"; // 未获取到分布式锁标志位，失败

        String surplus = "surplus";
        Object surplusObj = cache.get(surplus);
        Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

        if (surplusNum <= 0) return "fail"; // 库存数量不足，返回失败

        // ...执行逻辑操作
        surplusNum -= 1; // 对优惠券剩余数量 -1
        cache.set(surplus, surplusNum.toString()); // 更新库存 cache
    } finally {
        // 重点哦，只有在确定身份后才会执行删除
        if (Objects.equals(cache.get(lockKey), lockValue)) cache.del(lockKey); // 删除分布式锁
    }
    return "success";
}

```

大眼一看，上上下下实现了四版分布式锁，也该没问题了吧。

真相就是：解锁时，**由于判断锁和删除标志位并不是原子性的，所以可能还是会存在误删。**

1. 线程一获取锁后，执行流程 balabala... 判断锁也是自家的，这时 CPU 转头去做别的事情了，恰巧线程一的锁过期时间到了；
2. 线程二此时顺理成章的获取到了分布式锁，执行业务逻辑 balabala；
3. 线程一再次分配到时间片继续执行删除操作。

解决这种非原子操作的方式只能 **将判断元素值和删除标志位当作一个原子操作。**

5. 第五版 lua

很不友好的是，del 删除操作并没有提供原子命令，所以我们需要想点办法。

Redis 在 2.6 推出了脚本功能，允许开发者使用 Lua 语言编写脚本传到 Redis 中执行。

使用 Lua 脚本有什么好处呢？

1. 减少网络开销：原本我们需要向 Redis 服务请求多次命令，可以将命令写在 Lua 脚本中，这样执行只会发起一次网络请求。
2. 原子操作：Redis 会将 Lua 脚本中的命令当作一个整体执行，中间不会插入其它命令。
3. 复用：客户端发送的脚本会存储 Redis 中，其他客户端可以复用这一脚本而不需要使用代码完成相同的逻辑。

那我们编写一个简单的 Lua 脚本实现原子删除操作。

```
@GetMapping("/acquire")
public String acquireDiscount() {
    Jedis cache = new Jedis("localhost", 6379);
    cache.auth("123456");
    String lockKey = "lockKey"; // 设置锁标志位
    String lockValue = UUID.randomUUID().toString(); // 重点哦，设置客户端唯一标识
    try {
        String setResult = cache.set(lockKey, lockValue, "nx", "ex", 5); // 不存在即添加，存在返回 False
        if (!Objects.equals(setResult, "OK")) return "fail"; // 未获取到分布式锁标志位，失败

        String surplus = "surplus";
        Object surplusObj = cache.get(surplus);
        Integer surplusNum = Objects.isNull(surplusObj) ? 0 : Integer.parseInt(surplusObj.toString());

        if (surplusNum <= 0) return "fail"; // 库存数量不足，返回失败

        // ...执行逻辑操作
        surplusNum -= 1; // 对优惠券剩余数量 -1
        cache.set(surplus, surplusNum.toString()); // 更新库存 cache
    } finally {
        // 重点哦
        String script = "local cliVal = redis.call('get', KEYS[1]) " +
            "if(cliVal == ARGV[1]) then " +
            "redis.call('del', KEYS[1]) " +
            "return 'OK' " +
            "else " +
            "return nil " +
            "end ";
        cache.eval(script, Lists.newArrayList(lockKey), Lists.newArrayList(lockValue));
    }
    return "success";
}
```

重点就在 Lua 脚本这一块，重点说一下这块的逻辑。

script 脚本就是我们在 Redis 中执行的 Lua 脚本，后面跟的两个 List 分别是 KEYS、ARGV。

```
cache.eval(script, Lists.newArrayList(lockKey), Lists.newArrayList(lockValue));
```

- KEYS[1]: lockKey

- **ARGV[1]:** lockValue

代码不是很多，也比较简单，就是在 Java 中代码实现的逻辑放到了一个 Lua 脚本中。

```
# 获取 KEYS[1] 对应的 Val
local cliVal = redis.call('get', KEYS[1])
# 判断 KEYS[1] 与 ARGV[1] 是否保持一致
if(cliVal == ARGV[1]) then
    # 删除 KEYS[1]
    redis.call('del', KEYS[1])
    return 'OK'
else
    return nil
end
```

到了这种程度，已经可以放到一些并发量不大的项目中生产使用了。

待完成功能

虽然上述代码已经很大程度上解决了分布式锁可能存在的一些问题。

但是下述列出的问题部分就不是客户端代码范畴内的事情了：

- 如何实现可重入锁。
- 如何解决代码执行锁超时。
- 主从节点同步数据丢失导致锁丢失问题。

上述问题等下一篇介绍 Redisson 源码时会一一说明，顺道向大家推出一款 **Redis 官方推荐的客户端: Redisson**。

Jedis... 等客户端平常使用是绰绰有余的，但是在功能上还是和 Redisson 比不了。

并不是推荐一定要用 Redisson，根据不同场景选用不同客户端。

Redisson 就是为分布式提供 **各种不同锁以及多样化的技术支持**，感兴趣的小伙伴可以看一下 Github 上的介绍，挺详细的。

下一篇文章会详细介绍 Redisson 分布式锁的加锁、解锁原理。

看了 Redisson 的源码后更加深了对于分布式锁的了解。主要是对之前项目的分布式锁做了重构，在原有基础上增加了如下功能。

- 保证了加锁、解锁之间的原子性。
- 可重入的分布式锁。
- 分布式锁自动延期功能。

文末总结

本篇文章从最简单的分布式锁说起，一步一步的讲述存在的问题以及解决方式，最终得到个基本可用的分布式锁。

但是事无绝对，对于分布式锁的应用，还是推荐在 **代码以及数据库表中添加兜底策略，乐观锁等措施**。

更新: 2025-01-12 15:34:35

原文: <<https://www.yuque.com/magestack/12306/ag5pffwexihshe2s>>